

Vel Tech Group of Institutions

Name: UMA MAHESHWARI S

Email: vtu28408@veltech.edu.in

Roll no: vtu28408

Phone: null

Branch: VTU

Department: CSE with AIDS

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 5_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n , denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 1 5 3 2 6

Output: Given Array

4 1 5 3 2 6

After iteration 1

1 4 5 3 2 6

After iteration 2

1 4 5 3 2 6

After iteration 3

1 4 5 2 3 6

After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>
```

```
int iter = 1;
```

```
void printArray(int a[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", a[i]);  
}
```

```
void merge(int a[], int l, int m, int r, int n) {  
    int i = l, j = m + 1, k = 0;  
    int temp[20];
```

```
    while (i <= m && j <= r) {  
        if (a[i] < a[j])  
            temp[k++] = a[i++];  
        else  
            temp[k++] = a[j++];  
    }
```

```
    while (i <= m) temp[k++] = a[i++];  
    while (j <= r) temp[k++] = a[j++];
```

```
    for (i = l, k = 0; i <= r; i++, k++)  
        a[i] = temp[k];
```

```
    printf("After iteration %d ", iter++);  
    printArray(a, n);  
}
```

```
void mergeSort(int a[], int l, int r, int n) {  
    if (l < r) {  
        int m = (l + r) / 2;  
        mergeSort(a, l, m, n);  
        mergeSort(a, m + 1, r, n);
```

```
        merge(a, l, m, r, n);
    }
}
```

```
int main() {
    int n, a[20];
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Given Array ");
    printArray(a, n);

    mergeSort(a, 0, n - 1, n);

    printf("Sorted Array ");
    printArray(a, n);

    return 0;
} // You are using GCC
```

Status : Correct

Marks : 10/10

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n , representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Harish

7.8

23

ece

Trell

8

21

cse

Kamalesh

7

25

it

Maaran

5.6

20

civil

age

Output: Sorted Student Records:

Name	GPA	Age	Major
------	-----	-----	-------

Maaran	5.60	20	civil
--------	------	----	-------

Trell	8.00	21	cse
-------	------	----	-----

Harish	7.80	23	ece
--------	------	----	-----

Kamalesh	7.00	25	it
----------	------	----	----

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Student {  
    char name[20], major[20];  
    float gpa;  
    int age;  
} s[10];
```

```
char key[10];
```

```
int compare(struct Student a, struct Student b) {  
    if (strcmp(key, "gpa") == 0)  
        return a.gpa > b.gpa;  
    else if (strcmp(key, "age") == 0)  
        return a.age > b.age;  
    else  
        return strcmp(a.major, b.major) > 0;  
}
```

```
void swap(struct Student *a, struct Student *b) {  
    struct Student t = *a;  
    *a = *b;  
    *b = t;  
}
```

```
int partition(int low, int high) {  
    struct Student pivot = s[high];  
    int i = low - 1;  
  
    for (int j = low; j < high; j++) {  
        if (!compare(s[j], pivot)) {  
            i++;  
            swap(&s[i], &s[j]);  
        }  
    }  
    swap(&s[i + 1], &s[high]);  
    return i + 1;  
}
```

```
void quickSort(int low, int high) {  
    if (low < high) {  
        int pi = partition(low, high);  
        quickSort(low, pi - 1);  
        quickSort(pi + 1, high);  
    }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```

for (int i = 0; i < n; i++)
    scanf("%s %f %d %s", s[i].name, &s[i].gpa, &s[i].age, s[i].major);

scanf("%s", key);

quickSort(0, n - 1);

printf("Sorted Student Records: ");
printf("Name\t\tGPA\t\tAge\t\tMajor ");

for (int i = 0; i < n; i++)
    printf("%-10s\t%.2f\t%d \t\t%-15s ", s[i].name, s[i].gpa, s[i].age, s[i].major);

return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

banana pineapple mango chikoo jack star guava lemon tomato orange

Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void swap(char a[], char b[]) {  
    char temp[50];  
    strcpy(temp, a);  
    strcpy(a, b);  
    strcpy(b, temp);  
}
```

```
void heapify(char arr[][50], int n, int i) {  
    int largest = i;  
    int l = 2*i + 1;  
    int r = 2*i + 2;
```

```
    if (l < n && strcmp(arr[l], arr[largest]) > 0)  
        largest = l;
```

```
    if (r < n && strcmp(arr[r], arr[largest]) > 0)  
        largest = r;
```

```
    if (largest != i) {  
        swap(arr[i], arr[largest]);  
        heapify(arr, n, largest);  
    }  
}
```

```
void heapSort(char arr[][50], int n) {  
    for (int i = n/2 - 1; i >= 0; i--)  
        heapify(arr, n, i);
```

```
    for (int i = n - 1; i > 0; i--) {  
        swap(arr[0], arr[i]);  
        heapify(arr, i, 0);
```

```

    }
}

int main() {
    int n;
    scanf("%d", &n);

    char arr[20][50];

    for (int i = 0; i < n; i++)
        scanf("%s", arr[i]);

    heapSort(arr, n);

    for (int i = 0; i < n; i++)
        printf("%s ", arr[i]);

    return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8

8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n , representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x , representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x , as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

1 2 2 2 2 3 4 8 8 8

8

Output: 7 9

Answer

```
#include <stdio.h>
```

```
int first(int a[], int l, int r, int x) {  
    if (l > r) return -1;
```

```
    int mid = (l + r) / 2;
```

```

    if ((mid == 0 || x > a[mid - 1]) && a[mid] == x)
        return mid;
    else if (a[mid] < x)
        return first(a, mid + 1, r, x);
    else
        return first(a, l, mid - 1, x);
}

```

```

int last(int a[], int l, int r, int x, int n) {
    if (l > r) return -1;

    int mid = (l + r) / 2;

    if ((mid == n - 1 || x < a[mid + 1]) && a[mid] == x)
        return mid;
    else if (a[mid] > x)
        return last(a, l, mid - 1, x, n);
    else
        return last(a, mid + 1, r, x, n);
}

```

```

int main() {
    int n, a[20], x;
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    int f = first(a, 0, n - 1, x);
    int l = last(a, 0, n - 1, x, n);

    if (f == -1)
        printf("NO OCCURRENCES");
    else
        printf("%d %d ", f, l);

    return 0;
} // You are using GCC

```

Status : Correct

Marks : 10/10